Solutions to Assignment 1
Advanced Tools for Complex Network Analysis
CNET 5052, Spring 2026

Caleb Chandler*

1. *Question 1.*

(a) Since each node can only add m edges under the BA model, I determined the most likely m value by dividing the number of edges by the size of the network, and rounding to the nearest integer since m must be discrete.

(b) See notebook.

(c) For metrics, I chose assortativity, clustering, and path length. These work well for the BA model: BA networks tend to have a neutral or slightly negative assortativity, very low clustering, and a short average path length.

(d)
i. Assortativity: Both values are very unlikely under the ensemble, but in different directions; network 1 being too low and network 2 being too high. More of an outlier for 1 though.

ii. Clustering: Clustering is similar for both observed and ensemble. Fail to reject.

iii. Path Length: For network 1, path length is firmly in the region of plausibility, but for network 2 it is impossible.

iv. Verdict: The path length result alone is enough to disqualify network 2 as resulting from the BA process. Network 1 is a little iffy on assortativity, but not to an extent that's totally unreasonable. Given that it's a match for the other two, I will say network 1 is likely to have come from the model.

2. *Question 2.*

* **REPO FORK** https://github.com/caleb-chandler/cnet5052_sp26

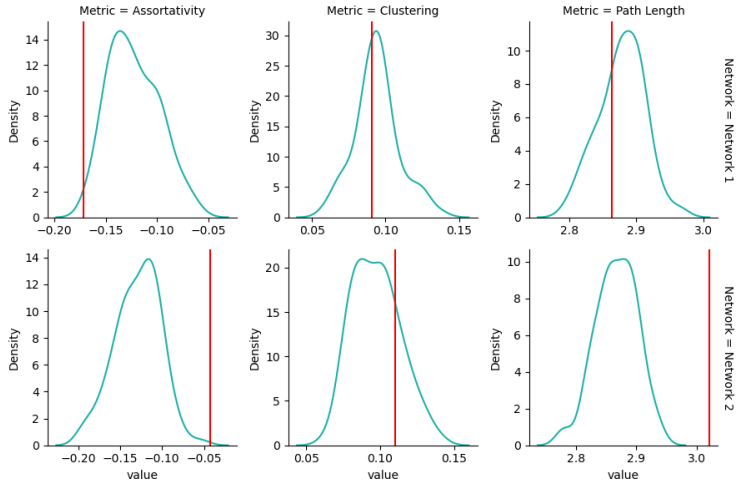

FIG. 1. Comparison of network metrics against the BA model ensemble.

(a) Modularity increases as size does, and appears to increase nonlinearly with decrease in m, since lower m means sparser graphs which naturally have more bottlenecks that modularity optimization can exploit to find spurious partitions. The reason this is damning for modularity is the same reason it is for ER graphs: there is no special process leading to the creation of communities, meaning modularity is "hallucinating" communities where they don't actually exist. These hallucinations are likely to be worse on larger graphs.
(AI used for plotting section)

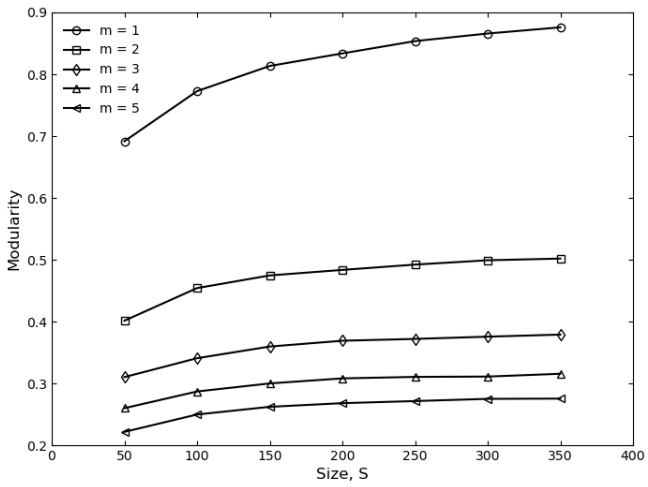

FIG. 2. Modularity scaling with network size and parameter m.

3. *Question 3.*

(a) i. See below.

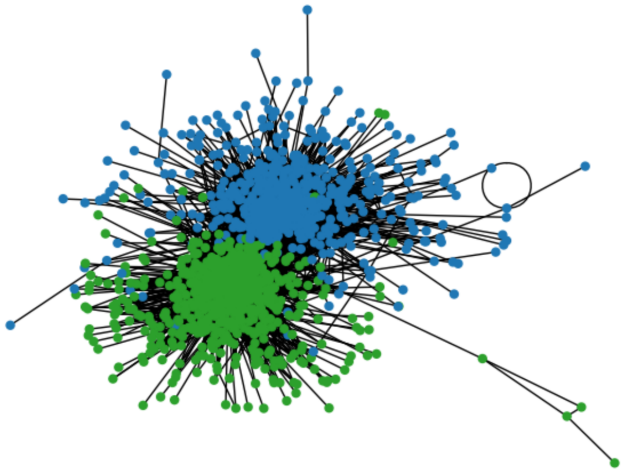

FIG. 3. Community structure detected by the Louvain algorithm.

ii. 0.427

iii. 11

iv. See notebook.

v. I have to run the notebook from a docker container and switch environments for this question, so I couldn't port the layout over from the previous question. The layout is still similar even if not exactly the same, so I hope that's enough. I used minimize_blockmodel_dl for community detection, which seems not to have done nearly as well as Louvain.

vi. See notebook.

vii. This is an excellent example of the pitfalls of modularity as a reliable metric: Louvain finds communities in both the null and real graph, while SBM correctly finds no communities in the random graph (despite behaving worse relative to the ground-truth partition as shown in part b).

viii. My answer to this question last semester was Adjusted Rand Index, so this time I'll go with something different: Adjusted Mutual Information, which is exactly what it sounds like: Mutual Information – the amount you know about one variable by knowing about another – adjusted for the fact that a high number of clusters can lead to high mutual information by chance, making it ideal for comparing partitions (like these ones) where the sizes of the communities are significantly different.

4. *Question 4.*

i. Consensus clustering is essentially a way of taking randomness out of the equation and arriving at a stable partition for a stochastic method by aggregating many partitions generated by that method. The consensus matrix is

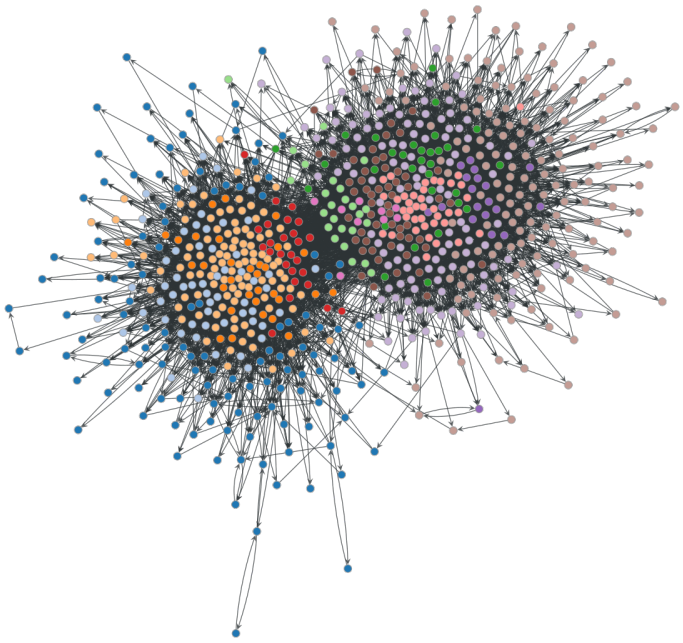

FIG. 4. Community structure detected using minimize_blockmodel_dl.

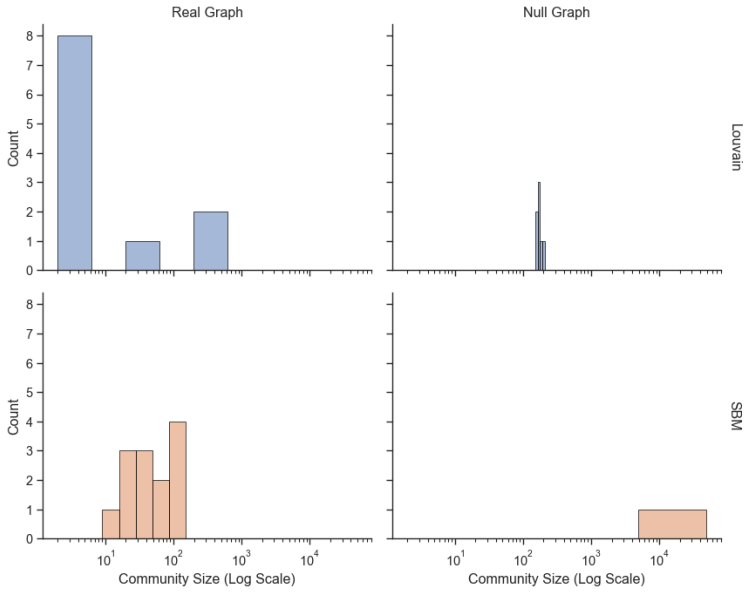

FIG. 5. Comparison of community sizes detected by Louvain versus SBM.

a matrix constructed from the cooccurrence of nodes; for every pair of nodes, their entry in the consensus matrix is the proportion of partitions in which they belong to the same community. [1]

ii. I put 100 connections between blocks and 500, 800, 1000, and 1200 for each of the within-block edges respectively.

iii. I wanted to use something new, and to make it easy on myself I also wanted to use something that was already in graph-tool, so I chose to use MCMC to sample partitions from the posterior distribution.

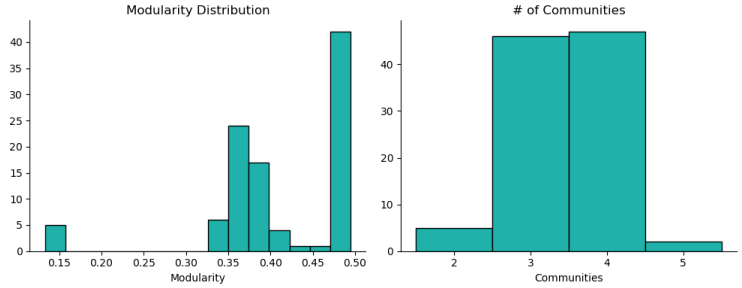

FIG. 6. Distribution of modularity and number of communities across runs for strong communities.

iv. I chose τ to be a cutoff at the top quartile. I chose this because I saw from my Google search that it was a common heuristic (and makes intuitive sense). I used minimize_blockmodel_dl once again with a fixed seed.

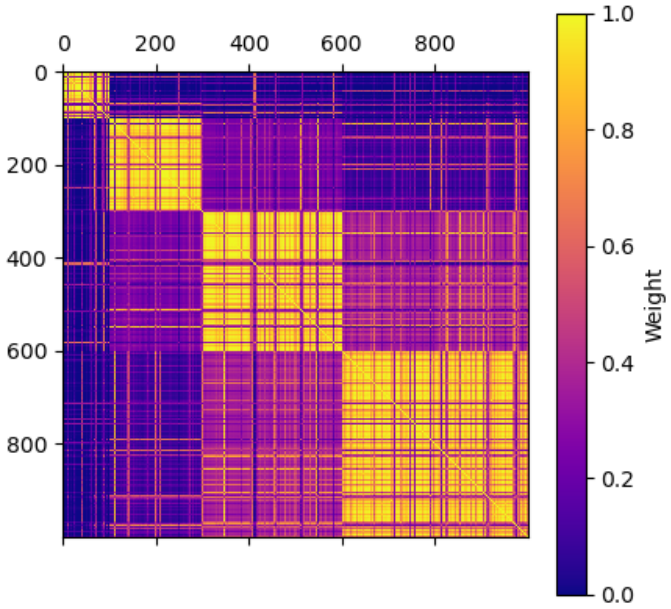

FIG. 7. Thresholded consensus matrix for strong communities.

v. I increased the between-community nodes by 50 and added increments of 100 to each within-community count.

vi. The consensus partition helped for the strong communities, but it completely failed on the weak ones. My best answer is that consensus clustering acts as an amplifier that improves the accuracy of an already generally-accurate algorithm, not a magic wand giving you the right answer every time. If the algorithm isn't working to begin with, it won't suddenly start working just because you increased the number of trials. Consensus clustering creates convergence, but there's no guarantee that this convergence will be in the right place. In this particular case, the MCMC method wasn't finding high-modularity partitions to begin with, and aggregating many trials just amplified the meaningless noise it was already generating.

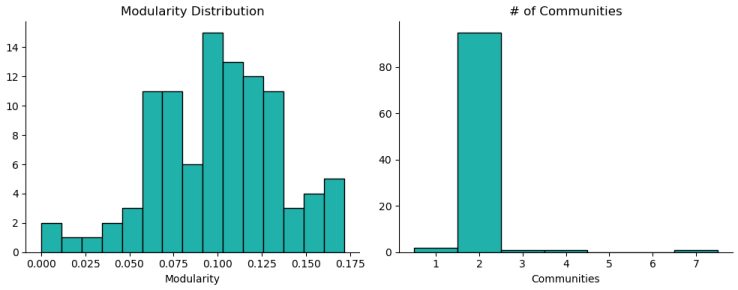

FIG. 8. Distribution of modularity and number of communities across runs for weak community structure.

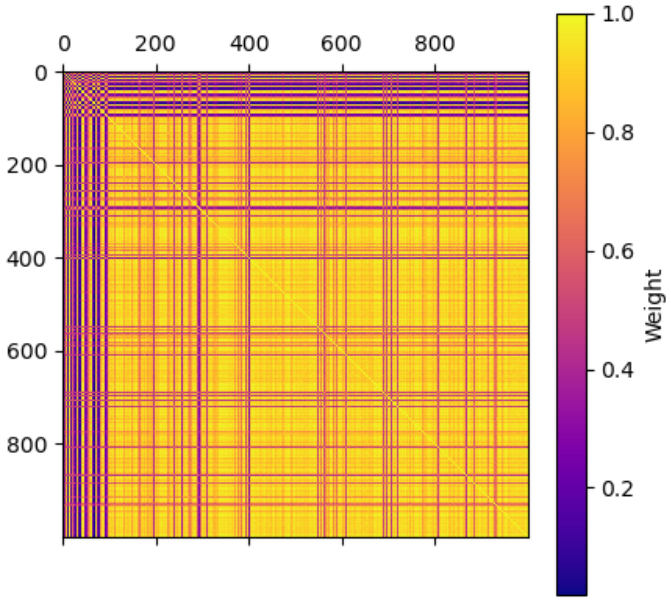

FIG. 9. Consensus matrix visualization for weak community structure.

[1] Lancichinetti, A., Fortunato, S. Consensus clustering in complex networks. *Sci Rep 2*, doi: 10.1038/srep00336.